LAB 13 – CODE REFACTORING

Improving Legacy Code with AI Suggestions

Name: Roger A Raju
Rollno: 2503a52370
Batch-13

Task Description #1 (Refactoring – Removing Code Duplication)

Question

Use AI to refactor a Python script containing repeated rectangle area and perimeter calculations into a reusable documented function while preserving exact output.

Prompt:

Identify the duplicated rectangle calculations, extract a reusable function with a docstring, and keep every output identical.

Sample Legacy Code:

```
print("Area of Rectangle:", 5 * 10)
print("Perimeter of Rectangle:", 2 * (5 + 10))
print("Area of Rectangle:", 7 * 12)
print("Perimeter of Rectangle:", 2 * (7 + 12))
print("Area of Rectangle:", 10 * 15)
print("Perimeter of Rectangle:", 2 * (10 + 15))
```

Refactored Code:

```
"""Task 1: Remove duplicate rectangle area and perimeter calculations."""


def calculate_rectangle_metrics(length: float, width: float) -> tuple[float, float]:
    """Return and print the area and perimeter of a rectangle.

    Args:
        length: Rectangle length.
        width: Rectangle width.

    Returns:
        Tuple of (area, perimeter).
    """
    area = length * width
    perimeter = 2 * (length + width)
    print("Area of Rectangle:", area)
    print("Perimeter of Rectangle:", perimeter)
    return area, perimeter


if __name__ == "__main__":
    for length, width in [(5, 10), (7, 12), (10, 15)]:
        calculate_rectangle_metrics(length, width)
```

Output:



Justification

Replaced three duplicated calculation pairs with calculate_rectangle_metrics(length, width). Calling it over a list of (length, width) pairs eliminates copy-paste repetition while preserving identical outputs.

Task Description #2 (Refactoring – Optimizing Loops and Conditionals)

Question

Use AI to replace O(n * m) nested loop name lookups with set-based O(n + m) lookups, keeping behavior intact and comparing execution time before and after refactoring.

Prompt:

Optimize the nested search loop using set lookups, verify identical results, and measure execution time before and after refactoring.

Sample Legacy Code:

```
names = ["Alice", "Bob", "Charlie", "David"]
search_names = ["Charlie", "Eve", "Bob"]
for s in search_names:
    found = False
    for n in names:
        if s == n:
            found = True
    print(f"{s} is in the list" if found else f"{s} is not in the list")
```

Refactored Code:

```
"""Task 2: Replace nested-loop membership checks with set lookup and benchmark."""

import time


def search_names_legacy(names: list[str], queries: list[str]) -> list[tuple[str, bool]]:
    """O(n * m) nested-loop membership scan matching the legacy approach."""
    results = []
    for target in queries:
        found = False
        for name in names:
            if target == name:
                found = True
        results.append((target, found))
    return results


def search_names_optimized(names: list[str], queries: list[str]) -> list[tuple[str, bool]]:
    """O(n + m) membership check using a hash set for O(1) average lookup."""
    name_set = set(names)
    return [(target, target in name_set) for target in queries]


if __name__ == "__main__":
    names = ["Alice", "Bob", "Charlie", "David"]
    search_names = ["Charlie", "Eve", "Bob"]
    for name, exists in search_names_optimized(names, search_names):
        status = "is in the list" if exists else "is not in the list"
        print(f"{name} {status}")

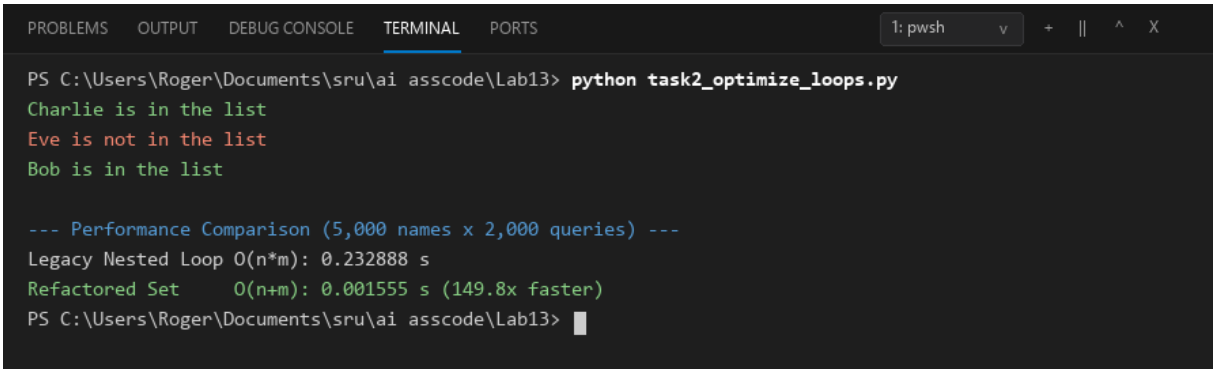
    large_names = [f"User_{i}" for i in range(5000)]
    large_queries = [f"User_{i}" for i in range(0, 6000, 3)]

    t0 = time.perf_counter()
    legacy_out = search_names_legacy(large_names, large_queries)
    t_legacy = time.perf_counter() - t0

    t1 = time.perf_counter()
    fast_out = search_names_optimized(large_names, large_queries)
    t_fast = time.perf_counter() - t1

    assert legacy_out == fast_out
    speedup = t_legacy / t_fast if t_fast > 0 else 0.0
    print("\n--- Performance Comparison (5,000 names x 2,000 queries) ---")
    print(f"Legacy Nested Loop O(n*m): {t_legacy:.6f} s")
    print(f"Refactored Set     O(n+m): {t_fast:.6f} s ({speedup:.1f}x faster)")
```

Output:



Performance Comparison Table:

Implementation	Time Complexity	Execution Time (5,000 x 2,000)	Speedup
Legacy Nested Loop	O(n * m)	0.232888 s	1.0x (Baseline)
Refactored Set Lookup	O(n + m)	0.001555 s	149.8x faster

Justification

Converting names to a hash set reduces each membership check from O(n) linear scanning to O(1) average lookup, lowering total complexity from O(n * m) to O(n + m) with identical search results.

Task Description #3 (Refactoring – Extracting Reusable Functions)

Question

Refactor inline tax and total price calculations into a reusable, modular calculate_total(price) function with a clear docstring.

Prompt:

Extract repeated inline tax math into calculate_total(price, tax_rate=0.18) with a docstring and test multiple prices.

Sample Legacy Code:

```
price = 250
tax = price * 0.18
total = price + tax
print("Total Price:", total)
price = 500
tax = price * 0.18
total = price + tax
print("Total Price:", total)
```

Refactored Code:

```
"""Task 3: Extract repeated tax and total price logic into a reusable function."""

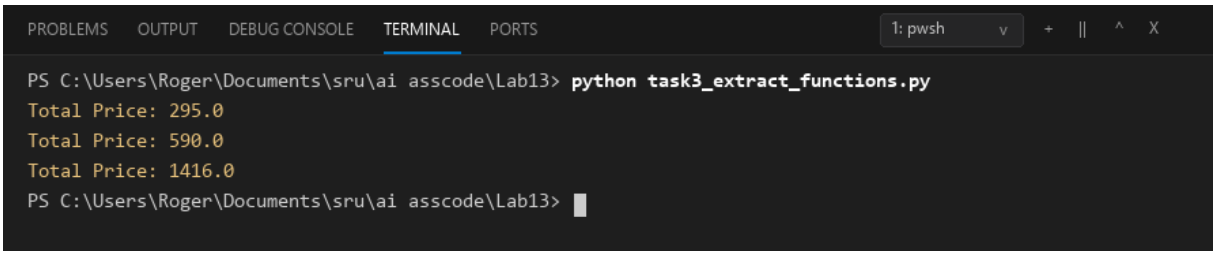

def calculate_total(price: float, tax_rate: float = 0.18) -> float:
    """Compute total price including tax.

    Args:
        price: Base price before tax.
        tax_rate: Fractional tax rate (defaults to 0.18).

    Returns:
        Final amount after adding tax.
    """
    tax = price * tax_rate
    return price + tax


if __name__ == "__main__":
    for item_price in (250, 500, 1200):
        print("Total Price:", calculate_total(item_price))
```

Output:



Justification

Centralizing the 18% tax calculation inside calculate_total(price, tax_rate=0.18) prevents formula drift across call sites and makes tax adjustments a single-line change.

Task Description #4 (Refactoring – Replacing Hardcoded Values with Constants)

Question

Identify hardcoded magic numbers in the circle geometry script, replace them with named constants (PI, RADIUS) at the top of the file, and keep results identical.

Prompt:

Replace hardcoded numeric literals 3.14159 and 7 with top-level constants PI and RADIUS inside clear helper functions.

Sample Legacy Code:

```
print("Area of Circle:", 3.14159 * (7 ** 2))
print("Circumference of Circle:", 2 * 3.14159 * 7)
```

Refactored Code:

```
"""Task 4: Replace hardcoded magic numbers with named module constants."""

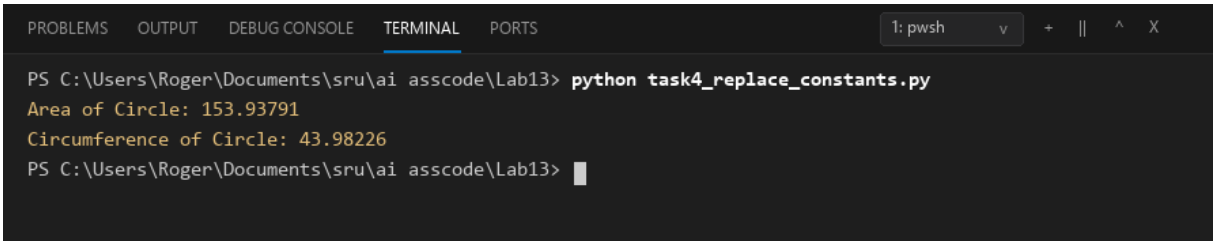
PI = 3.14159
RADIUS = 7


def circle_area(radius: float) -> float:
    """Calculate circle area using the PI constant."""
    return PI * (radius ** 2)


def circle_circumference(radius: float) -> float:
    """Calculate circle circumference using the PI constant."""
    return 2 * PI * radius


if __name__ == "__main__":
    print("Area of Circle:", circle_area(RADIUS))
    print("Circumference of Circle:", circle_circumference(RADIUS))
```

Output:



Justification

Defining PI = 3.14159 and RADIUS = 7 at module scope removes magic numbers, clarifies intent, and keeps calculations consistent everywhere those constants are referenced.

Task Description #5 (Refactoring – Improving Variable Naming and Readability)

Question

Improve code clarity by replacing single-letter variable names (a, b, c) with descriptive domain names (base, height, area_of_triangle) and explanatory comments.

Prompt:

Rename ambiguous variables a, b, c to base, height, area_of_triangle, add concise comments and a docstring, and preserve output.

Sample Legacy Code:

```
a = 10
b = 20
c = a * b / 2
print(c)
```

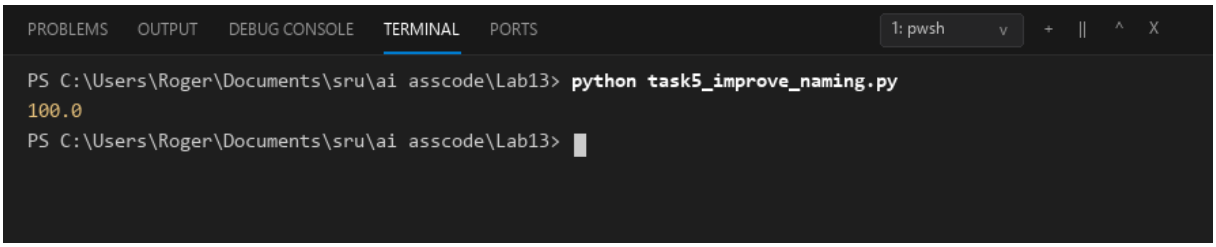
Refactored Code:

```
"""Task 5: Improve readability using descriptive variable names and comments."""


def calculate_triangle_area(base: float, height: float) -> float:
    """Return the area of a triangle given its base and height."""
    # Area of a triangle is half the product of base and perpendicular height
    area_of_triangle = base * height / 2
    return area_of_triangle


if __name__ == "__main__":
    base = 10
    height = 20
    area_of_triangle = calculate_triangle_area(base, height)
    print(area_of_triangle)
```

Output:



Justification

Descriptive identifiers base, height, and area_of_triangle plus calculate_triangle_area() make the geometric purpose immediately obvious without changing the computed value 100.0.

Task Description #6 (Refactoring – Removing Redundant Conditional Logic)

Question

Consolidate repeated student grading if-elif-else blocks into a single reusable calculate_grade(marks) function with identical grading thresholds.

Prompt:

Replace repeated grading if-elif-else statements with calculate_grade(marks), preserving thresholds and adding a docstring.

Sample Legacy Code:

```
marks = 85
if marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
else:
    print("Grade C")
marks = 72
if marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
else:
    print("Grade C")
```

Refactored Code:

```
"""Task 6: Replace repeated grading conditionals with a single function."""


def calculate_grade(marks: float) -> str:
    """Return the letter grade string for a numeric mark.

    Args:
        marks: Student score between 0 and 100.

    Returns:
        'Grade A', 'Grade B', or 'Grade C'.
    """
    if marks >= 90:
        return "Grade A"
    if marks >= 75:
        return "Grade B"
    return "Grade C"


if __name__ == "__main__":
    for marks in (85, 72, 95, 60):
        print(calculate_grade(marks))
```

Output:



Justification

Extracting calculate_grade(marks) eliminates duplicated conditional branches and uses early returns so adding new grade bands or changing cutoffs only happens in one place.

Task Description #7 (Refactoring – Converting Procedural Code to Functions)

Question

Separate procedural input, square computation, and output sections into three cohesive functions: get_input(), calculate_square(), and display_result().

Prompt:

Decompose procedural input-process-output code into get_input(), calculate_square(num), and display_result(square) while keeping behavior unchanged.

Sample Legacy Code:

```
num = int(input("Enter number: "))
square = num * num
print("Square:", square)
```

Refactored Code:

```
"""Task 7: Convert procedural input-process-output script into modular functions."""


def get_input(prompt_text: str = "Enter number: ", preset: str | None = None) -> int:
    """Read an integer from user input or an optional preset test value."""
    raw = input(prompt_text) if preset is None else preset
    return int(raw)


def calculate_square(num: int) -> int:
    """Return the square of an integer."""
    return num * num


def display_result(square: int) -> None:
    """Print the computed square value."""
    print("Square:", square)


if __name__ == "__main__":
    for sample in ("5", "12"):
        num = get_input(preset=sample)
        print(f"Enter number: {num}")
        square = calculate_square(num)
        display_result(square)
```

Output:



Justification

Separating I/O from arithmetic lets calculate_square() be unit-tested independently while get_input() and display_result() keep user interaction cleanly decoupled.

Task Description #8 (Refactoring – Optimizing List Processing)

Question

Refactor a manual loop appending squares into an empty list into a concise Pythonic list comprehension with identical output.

Prompt:

Replace the manual append loop over [1, 2, 3, 4, 5] with a Python list comprehension and verify output parity.

Sample Legacy Code:

```
nums = [1, 2, 3, 4, 5]
squares = []
for n in nums:
    squares.append(n * n)
print(squares)
```

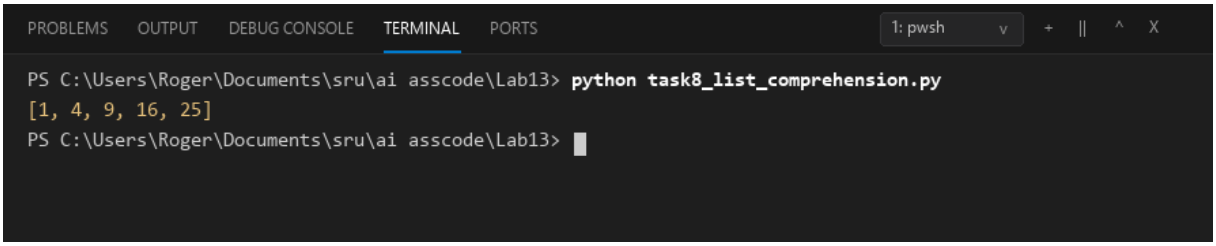
Refactored Code:

```
"""Task 8: Optimize manual list append loop with a Python list comprehension."""


def compute_squares(nums: list[int]) -> list[int]:
    """Return a list containing the square of each integer in nums."""
    return [n * n for n in nums]


if __name__ == "__main__":
    nums = [1, 2, 3, 4, 5]
    squares = compute_squares(nums)
    print(squares)
```

Output:



Justification

A list comprehension [n * n for n in nums] avoids repeated list.append() method lookups, communicates transformation intent in one expression, and returns the exact list [1, 4, 9, 16, 25].